



Quantumons: Q-Battle

Integrating Quantum Mechanics into Interactive Gaming

Manan Gupta¹, Rohan Khanna², Peng Shen³, Peter Wu⁴ - Cluster 2, 2025

The Harker School¹, Northwood High School², Woodbridge High School³, Northwood High School⁴



MOTIVATION

Current approaches to quantum education rely on static visualizations or mathematical formulas that fail to teach effectively. The solution is an appealing, easy-to-understand game. Using Qiskit, an open-source quantum toolkit, we integrated quantum principles in our code. Our approach uses a character-driven video-game design where each character (a 'Quantumon') has unique quantum abilities, such as waveform manipulation, quantum tunneling effects, and entanglement effects that directly participate in classical game mechanics including damage calculation, turn management, and visual feedback via their own "qubit" to control their behavior. The system's modular architecture, featuring separate quantum move implementations and ability systems for each character, makes rapid prototyping of new quantum mechanics very easy to learn quantum concepts in an enjoyable manner.

About QISKIT

Our Quantumons project leverages IBM's Qiskit framework to translate abstract quantum computing concepts into tangible gameplay mechanics. Unlike classical computing's binary bits (0 or 1), quantum computing employs qubits that can exist in superposition states—simultaneously representing multiple values until measured. Qiskit enables us to construct quantum circuits that drive character-specific mechanics: Bitzy utilizes Hadamard gates for probabilistic lightning attacks, Neutrinette employs entanglement to create shared quantum states between characters, Resona harnesses quantum interference through waveform stacking for amplified damage, and Higsicrozma implements quantum tunneling via barrier systems that affect damage reduction and amplification. Through Qiskit's circuit execution, each character's moves generate probabilistic outcomes based on quantum measurements, allowing players to experience fundamental quantum phenomena—superposition, entanglement, interference, and tunneling—in an intuitive, game-driven context. This integration transforms complex quantum theory into accessible, interactive experiences that enhance player understanding of quantum computing principles while maintaining engaging gameplay mechanics.



Discussion

Our game begins with the player choosing one of the four available characters: Bitzy, Neutrinette, Resona, and Higsicrozma. Each character has its own distinct ability inspired by real quantum mechanics: superposition, entanglement, interference, and quantum tunneling, respectively. After the player has selected a character to play as, the player will play against an AI-controlled opponent, who is a boss named Singulon. The battle would be structured in a turn-based format, where the player selects quantum-based abilities available for the character that they selected to defeat the opponent.

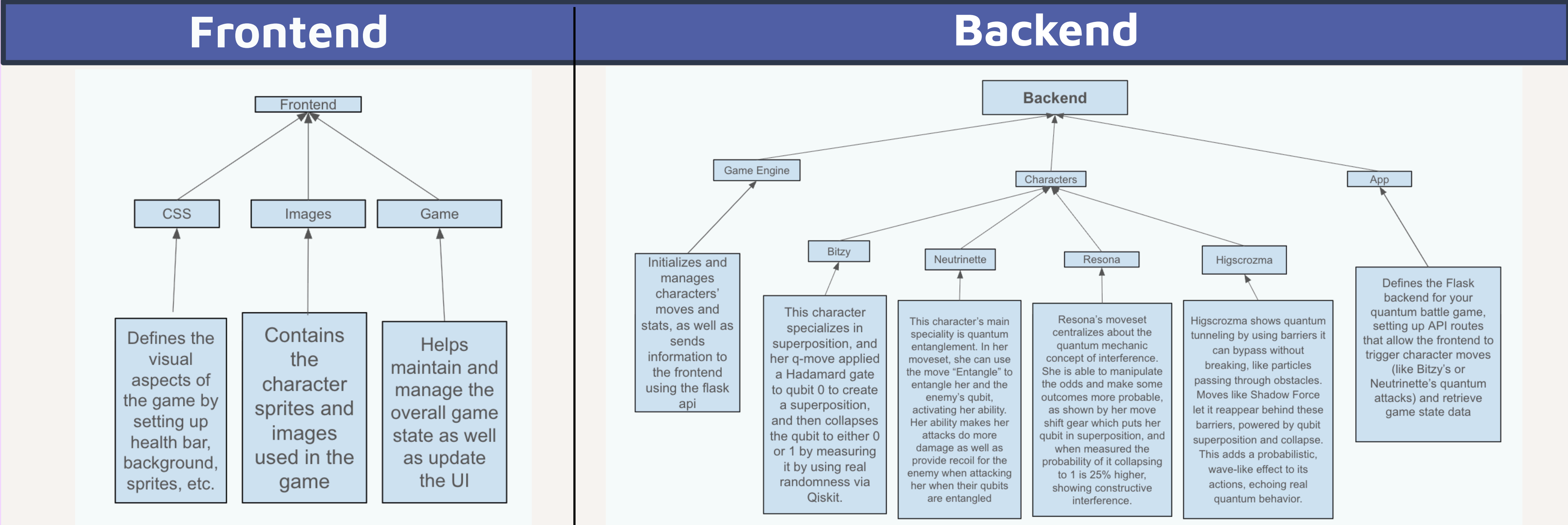
References

IBM Quantum Learning, Basics of Quantum Information, <https://quantum.cloud.ibm.com/learning/en/courses/basics-of-quantum-information>

IBM Qiskit Documentation, <https://qiskit.qotlabs.org/>

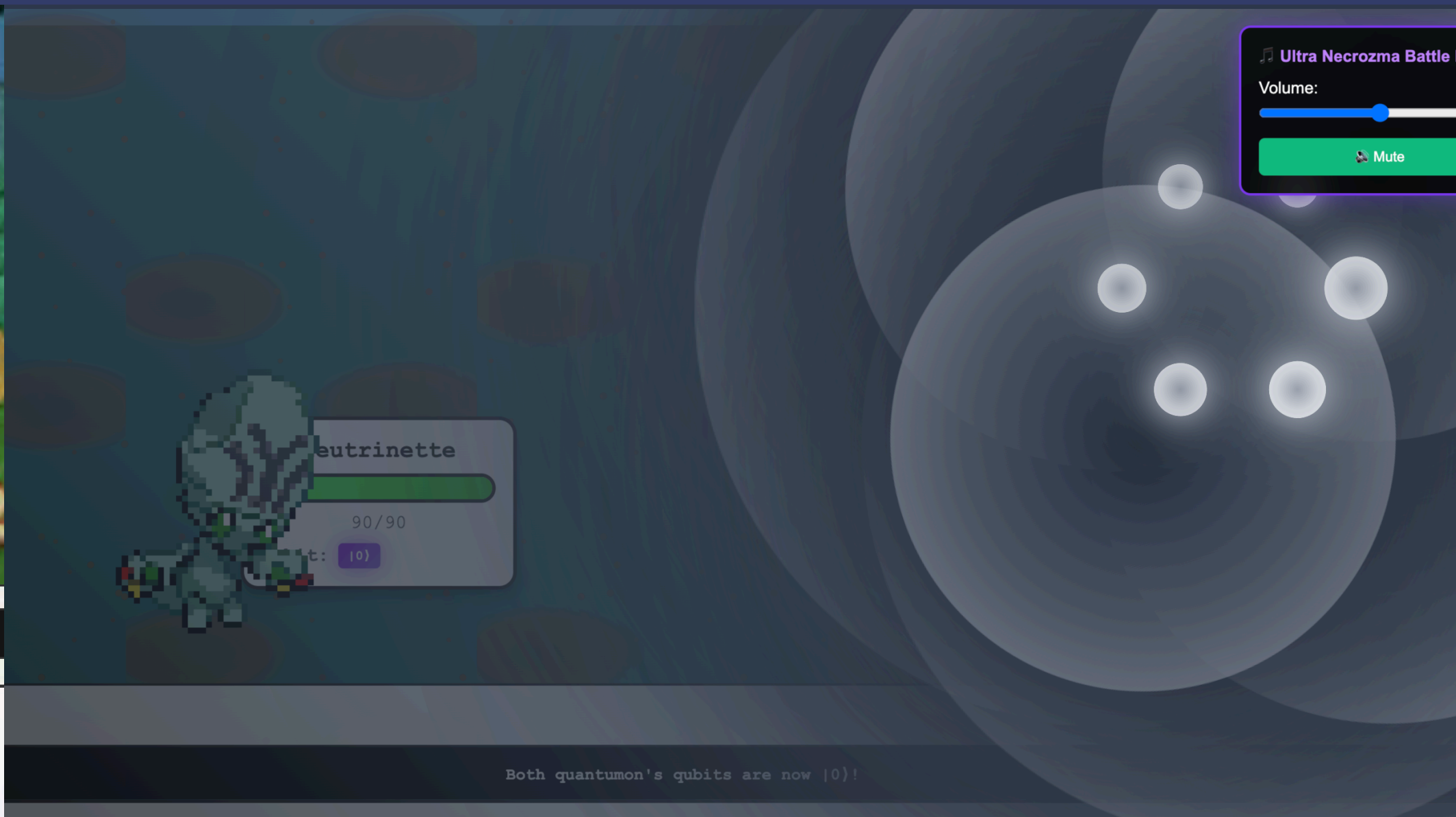
<https://github.com/mnn31/q-battle>

Methodology



<https://github.com/mnn31/q-battle>

Final Outcome



Resona using Q-METRONOME

Your Stats - ATK: 60, DEF: 85, SPD: 8
Singulon's HP: 400
Singulon's Stats - ATK: 60, DEF: 10, SPD: 5

Your move choices are:

1. Q-THUNDER
Bitzy's Q-Move. If the qubit is in a state of SUPERPOSITION, this move deals massive damage and collapses the qubit randomly. Else, fails. (DMG: 90)
2. SHOCK
Deals damage. Additional damage is dealt if the qubit and the enemy's qubit are in different states. (DMG: 30 + 20)
3. DUALIZE
Puts the qubit in a state of SUPERPOSITION if it wasn't previously.
4. BIT-FLIP
Flips the state of the enemy's qubit.

Debug Console showing Battle Logs

<https://github.com/mnn31/q-battle>

Architecture

Our Quantumons project employs a client-server architecture built with Flask for the backend and vanilla JavaScript for the frontend, creating a responsive web-based gaming experience. The backend utilizes Python with Qiskit integration to handle quantum circuit execution, game state management, and character-specific mechanics. The frontend implements a modular design with separate components for character selection, battle interface, and real-time animations. The system features a RESTful API architecture where the Flask backend serves game routes, processes quantum moves through Qiskit circuits, and maintains persistent game state, while the JavaScript frontend handles user interactions, real-time UI updates, and visual feedback. The architecture supports dynamic content delivery through static file serving for sprites, audio, and CSS assets, with the quantum mechanics layer abstracted through character-specific move implementations that translate quantum measurements into gameplay outcomes. This separation of concerns enables scalable development while maintaining real-time responsiveness for quantum circuit execution and visual feedback systems.

Acknowledgements

We would like to express our sincere gratitude to our professors, Dr. Albert Siryaporn and Dr. Luis Jauregui, for their insightful lectures and invaluable support in addressing theoretical questions throughout our project. We extend special thanks to our Teacher Fellow, Mr. Andrew Gibas, for organizing our daily lectures and lab sessions, and to our collaborator Izzy for assisting with initial resources and project planning. Lastly, we deeply appreciate the UCI COSMOS program for offering us this exceptional opportunity to explore quantum computing through such an innovative project.